UGR LINES

AI Engineer Technical Case

AI Booking Intake & Operations Assistant

Objective

Build a small working prototype that turns incoming booking e-mails into structured booking records, presents an interactive operational dashboard, and allows users to query the booking data in natural language. The goal is to demonstrate how you think, learn, implement, test and explain an AI-enabled business workflow.

What we are evaluating

- Problem understanding and prioritization
- Python and software engineering fundamentals
- Practical use of LLM / agent capabilities
- Safe handling of incomplete, malformed and ambiguous data
- Testing and verification mindset
- Clear and honest technical communication

Important

This is not a feature-count exercise. A stable core flow is more valuable than many unfinished features. If something is incomplete, explain what is done, what remains, why it remains and what you would do next.

1. Business Scenario

Booking requests arrive by e-mail in different formats. An operations user may need to read the e-mail, identify the booking details, validate the information and enter a booking manually. Your task is to automate this intake flow with a controlled AI-based assistant.

Target flow

E-mail → AI processing → structured extraction → validation → booking record → interactive dashboard + natural-language query

2. Core Requirements

2.1 E-mail processing

- Process the booking e-mails supplied in the separate data package.
- Support different writing styles and layouts.
- A live mailbox integration is not required. Local files or another simple input method are sufficient.

2.2 Booking extraction

Extract the booking information that can be reliably identified. At minimum, consider:

Customer	Port of Loading (POL)	Port of Discharge (POD)
Cargo / vehicle information	Quantity	Requested date
Contact / reference information	Vessel / voyage when present	Source e-mail identifier

2.3 Validation and safe behavior

- Do not silently invent missing information.
- Flag missing, invalid or ambiguous fields for review.
- Normalize common formatting differences only when the correction is safe.
- A malformed e-mail must not stop processing of the remaining inputs.
- Keep a clear processing status for each booking.

Safety rule

If the system is not confident that a value is correct, it should ask for validation or mark the field as unresolved instead of guessing.

3. Agent / Tool Workflow

The solution should demonstrate at least one controlled AI-to-tool interaction. You may use an agent framework, structured tool calling, or a lightweight custom orchestration. Use the simplest design you can justify.

Example capability	Purpose
lookup_customer()	Check whether the customer can be resolved before booking creation.
create_booking()	Create a validated booking draft or mock booking record.
update_booking()	Apply a correction or follow-up to an existing booking.

These names are examples only. A mock implementation is enough. We want to see how you control the model before it creates or changes business data.

4. Data Persistence, Interactive Dashboard and Chatbot

4.1 Booking storage

- Persist the resulting booking records in SQLite, PostgreSQL or another simple store.
- Keep the data model clear enough to support later queries.
- Preserve the source e-mail reference for traceability.

4.2 Interactive Dashboard

Provide a simple interactive dashboard using the booking data created by the system.

The dashboard should give users a quick operational overview and should reflect the current stored booking data.

At minimum, include:

- Total number of bookings
- Bookings by processing status
- Bookings requiring validation
- Basic breakdown by customer, POL, POD or another meaningful operational dimension
- Simple filtering or interactive exploration

The dashboard does not need complex visual design. We are interested in how you select useful information and connect it to the underlying booking data.

4.3 Natural-language query

Provide a simple chatbot or natural-language query interface that answers questions using the stored booking data.

The chatbot and dashboard should work on the same booking data. The user should be able to ask operational questions and receive a clear answer based on the available records.

Where relevant, the application should also reflect the requested information in the dashboard through filtering, highlighting or an updated summary.

The chatbot should not answer operational questions using unrelated model knowledge when the required data is not available.

“How many bookings does ABC Shipping have?”

“Show bookings with Izmir as the port of loading.”

“Which bookings require validation?”

“Summarize the latest three bookings.”

5. Input Data Package

The booking e-mail dataset is not included in this document. It will be provided separately as the case input package.

- Expect multiple customers and repeated booking requests.
- Some inputs will be complete, and some will contain missing, malformed, noisy or ambiguous information.
- Follow-up, correction or cancellation messages may be included.
- Treat the supplied data as case material only.

Data handling

Do not place confidential or sensitive case data in a public repository. If the provided package is marked restricted, keep it outside the repository and document how the application expects the data to be supplied.

6. Technical Expectations

Language	Python
Repository	GitHub repository with meaningful commits
Code	Readable, modular structure with clear responsibilities
Tests	Automated tests for the critical rules and failure paths. TDD is encouraged, not required.
Secrets	No API keys or credentials in source code
Dependencies	requirements.txt, pyproject.toml or equivalent
LLM / AI	Any suitable provider, model or framework may be used
AI coding tools	Allowed. Be ready to explain where and how they were used
Reproducibility	A reviewer should be able to run the project by following the README

Not required

- Real UGR ERP integration
- Production mailbox access
- Cloud deployment
- Complex frontend design
- Production-grade authentication and authorization

7. Deliverables

Deliverable	Expectation
GitHub repository	Source code, tests and dependency definitions
README	Setup, run instructions, assumptions, architecture summary and known limitations
Architecture view	A simple diagram or flow showing components and data movement

Deliverable	Expectation
Working demo	Core e-mail → booking → interactive dashboard + chatbot flow
Presentation	5-10 minutes, focused on decisions, result, limitations and next steps

8. Definition of Done

- The supplied e-mails can be processed without manual editing of each input.
- Valid requests produce structured booking records.
- Missing, invalid or ambiguous values are visible and are not silently invented.
- Booking data is persisted and traceable to its source e-mail.
- The dashboard provides an interactive operational summary of the stored booking data.
- The user can filter or explore booking information through the dashboard.
- The chatbot can answer basic operational questions using the same stored booking data.
- Dashboard and chatbot results remain consistent with the underlying booking records.
- Critical rules and at least one failure path are covered by automated tests.
- The project can be started by following the README.

9. Presentation

Keep the presentation short. We want to understand your decisions, not see a long slide deck.

- Where did you start and what did you prioritize?
- What architecture did you choose and why?
- What did you complete?
- What is incomplete and why?
- Where did AI tools help you?
- How did you verify AI-generated code or output?
- What would you change before production?
- If you had one more day, what would you do first?

10. Optional Enhancements

Only attempt these after the core flow is stable.

- Measure end-to-end response time and identify latency sources.
- Handle follow-up e-mails as updates to an existing booking.
- Add confidence scoring or an explicit human-approval step.
- Add structured logging or lightweight observability.
- Containerize the application.

11. Evaluation Focus

Problem framing	Did you understand the business problem and choose a sensible scope?
Architecture	Are the workflow, responsibilities and AI boundaries clear?
Implementation	Does the core flow work and is the code understandable?
Robustness	How are incomplete, malformed and ambiguous inputs handled?
Testing	Are important rules and failure cases verified?
Transparency	Can you state clearly what worked, what did not and why?
Communication	Can you explain the solution briefly and answer questions about your own decisions?

Final note

We do not expect a perfect product. We expect a clear, honest and technically reasoned prototype. If something does not work, show what you learned, where it failed and how you would continue.